极客时间

OpenClaw 入门

让 AI Agent 住进聊天软件

第 4 周 · 第 13 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

极客时间

灵魂拷问

V3 已经生产级了
但只有开发者能用

V4 目标：任何人通过手机聊天就能用你的 AI Agent

OpenClaw 在传统 CI/CD 之上可以加些什么？

极客时间

层次	V2（cron 自动化）	V4（OpenClaw）	体感区别
采集	pipeline.py 定时跑	同左	—
查询	你自己 cat *.json	用户发 /search MCP Bot 搜给你	数据从「死的」变成「活的」
分发	文件躺在磁盘	自动推 Telegram/飞书 三格式转换	知识主动找到用户
订阅	没有	/subscribe rag 新文章自动推给你	从拉模式变推模式

极客时间

一句话区别

极客时间

V2 采集自动化 = 数据进来了，死的
V4 服务自动化 = 数据活了，用户能查、能订、能收推送

OpenClaw 不是替代 cron ， 它在 cron 之上加了交互层、分发层、订阅层

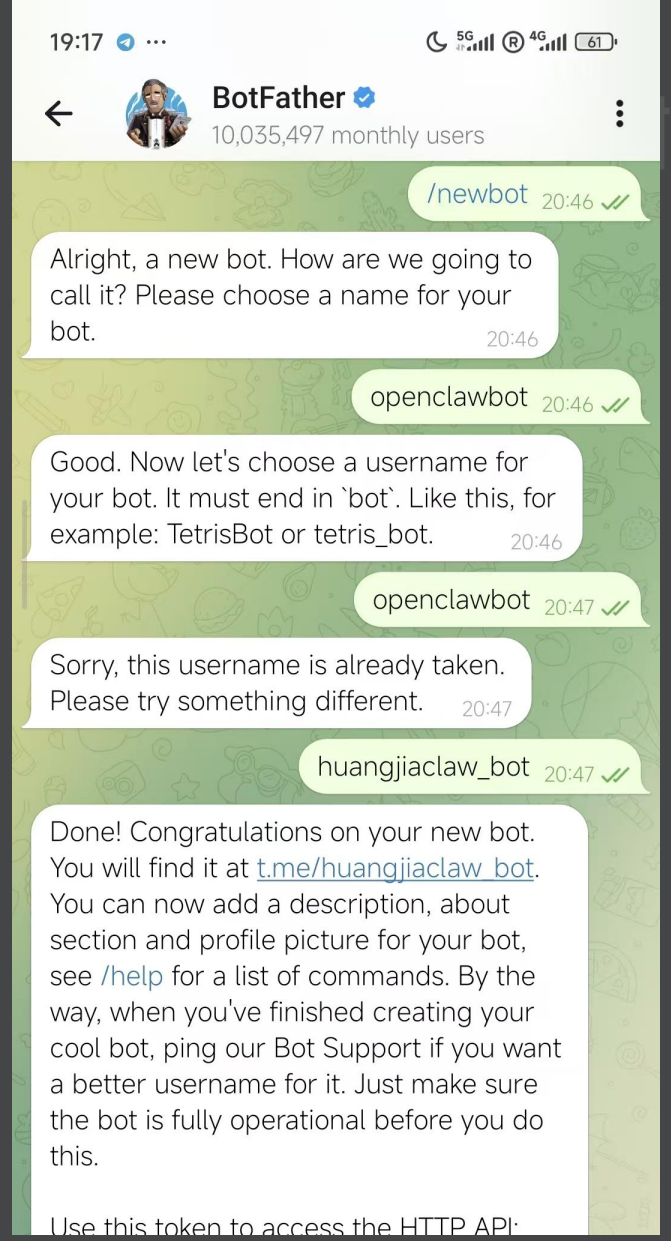
本节直播流程 · 你今天要交付什么

今天主线 30 分钟

- ① 装 OpenClaw + onboard(默认全局)　　实操 1
- ②创建 Telegram Bot + 接入 channel　　实操 1
- ③ 把 workspace 切到 v4-production/openclaw/　实操 2

完成标志(你下课时手机里要有的)

- Telegram Bot 收到「你好」回「你好」
- Bot 能答出「知识库现在有多少条记录」



极客时间黄佳老师出品

我的 OpenClaw 现在是这样

openclaw daemon start

terminal | bash

```
# ① daemon 在跑吗?
$ openclaw daemon status
  PID 18472 · uptime 4h 21m · port 18789

# ② Telegram 渠道接通了吗?
$ openclaw channels list
  telegram   my_kb_assistant_bot      Online

# ③ 模型默认是啥?
$ openclaw capability model auth status | head -5
  defaultModel: deepseek/deepseek-chat
  authProfile:  deepseek:default

# ④ 跑一次模型
$ openclaw capability model run \
      --model deepseek/deepseek-chat --prompt "回复 OK"
  provider: deepseek · model: deepseek-chat · OK

# ⑤ 配置 + daemon + channel + auth
$ openclaw doctor
      config valid ·     daemon up ·     telegram online ·     auth ok
```

我的 Telegram Bot 长这样

- 一个属于你自己的 Bot,你给它发什么它都听得懂、都答得上来。
- 它背后是 v4 知识库 + DeepSeek 模型 + OpenClaw 网关——三件套

极客时间

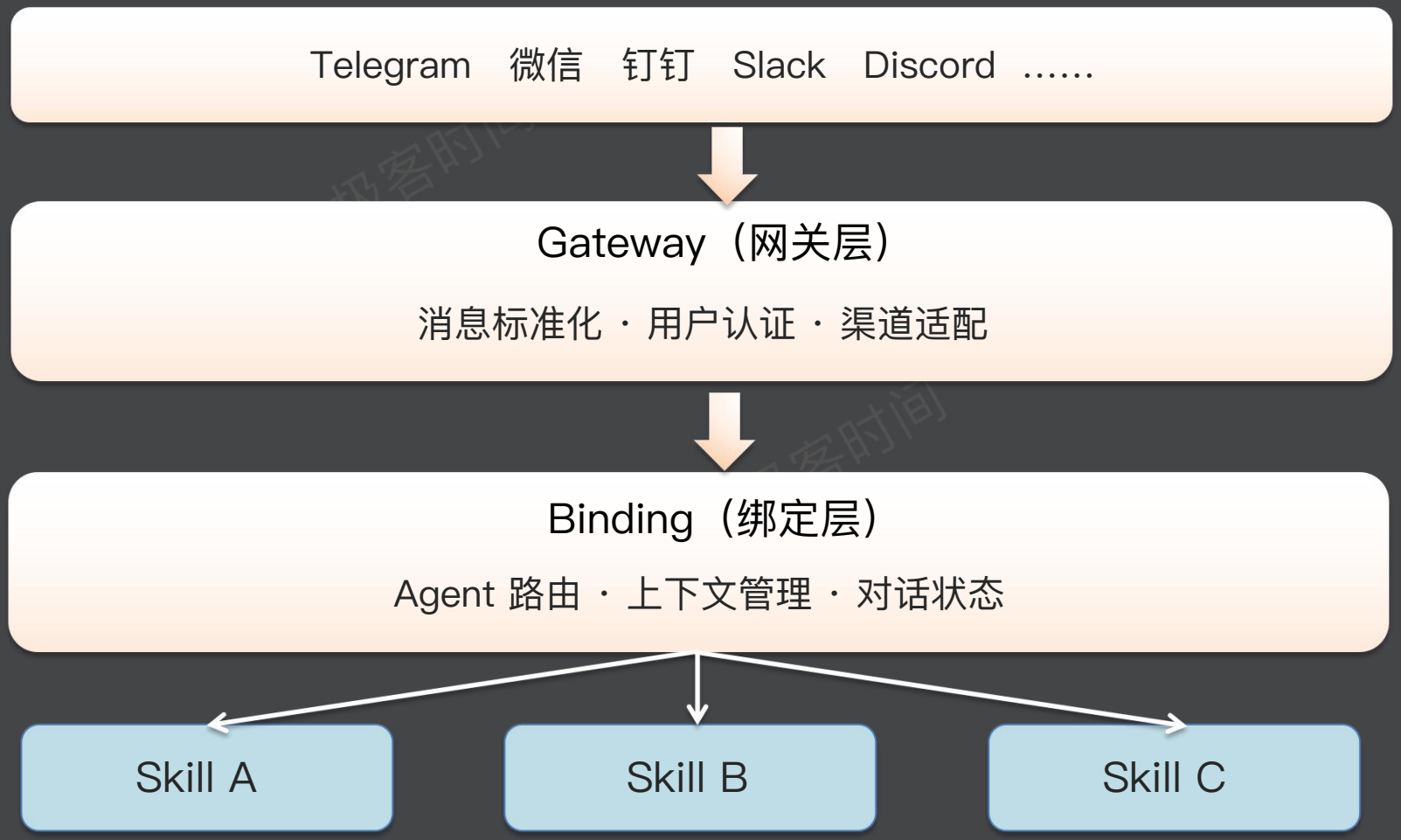
极客时间

OpenClaw 是什么

开源 AI Agent 运行时 · 145K+ GitHub Stars

极客时间黄佳老师出品

OpenClaw 三层架构



（按需加载，用完释放）

极客时间黄佳老师出品

Gateway 内部：消息处理完整链路

极客时间

用户发消息

↓

Gateway (WebSocket · Port 18789)

① 渠道适配　Telegram/微信/Slack → 统一消息格式

② 身份认证　DM Pairing · 用户白名单校验

③ 路由决策　Binding 规则匹配 → 选定 Agent

④ 上下文注入　Active Memory 自动拉取历史摘要

⑤ Skill 调度 description 匹配 → 按需加载 SKILL.md

⑥ LLM 调用　OpenAI 兼容接口 / 国产模型均可

⑦ 响应回写　格式适配 → 回到对应渠道

关键设计：步骤 ④ Active Memory = 新特性，不需手动管理上下文

OpenClaw 2026 最新进展

极客时间

- **Active Memory Plugin（记忆子 Agent）**
- 专用 Memory 子 Agent 在每次对话前自动拉取历史摘要
- 无需手动管理上下文——跨会话记忆开箱即用

- **OpenAI 兼容接口**
- /v1/models · /v1/embeddings · /v1/chat/completions
- 可将 OpenClaw 作为 OpenAI 代理，现有代码零改动切换国产模型

- **安全加固**
- 拦截 MAVEN_OPTS / SBT_OPTS 等 JVM 注入攻击
- 跨构建工具链环境变量注入防护

- **模型提供商更新**
- 原生支持 GPT-5 系列 · DeepSeek V3 · Qwen 2.5 Ultra

架构哲学之争：Gateway-First vs Agent-First

极客时间

OpenClaw（网关优先）

核心：中央 WebSocket 网关 (Port 18789)

设计哲学：平台管理所有渠道 + Agent

强项：50+ 渠道原生支持，会话路由

技能系统：SKILL.md（声明式）

记忆：Active Memory 子 Agent 注入

适合场景：多渠道 · 企业 Bot · 服务化

GitHub: 145K （2025 年末快速崛起）

Hermes Agent（代理优先）

核心：自我改进的 Agent 运行时

设计哲学：Agent 从经验中提取可复用 Skill

强项：自学习 · 失败后自动生成修复脚本

技能系统：成功案例 → 自动写入 skills/

记忆：内置 brain 文件持续精炼

适合场景：开发辅助 · 本地自主 Agent

GitHub: 22K （Nous Research, 2026.02）

极客时间

安装与配置

从 npm install 到 Agent 上线

极客时间

极客时间

安装 OpenClaw

terminal | bash

```bash
# 1. 确认 Node.js 版本（需要 22+）
node --version

# 如果低于 22：
nvm install 22 && nvm use 22

# 2. 全局安装 OpenClaw
npm install -g openclaw@latest

# 3. 验证安装
openclaw --version

# 4. 初始化项目（含守护进程）
cd ~/ai-knowledge-base
openclaw onboard --install-daemon

# onboard 会创建：
#   ~/.openclaw/openclaw.json5 (配置文件，支持注释)
#   ./AGENTS.md                (能力定义)
#   ./SOUL.md                  (性格定义)
```

```bash
# 真实生成:
#   ~/.openclaw/workspace/      ← 默认 workspace
#     ├ BOOTSTRAP.md   首次启动检查
#     ├ HEARTBEAT.md   健康检查指引
#     ├ IDENTITY.md    agent 名字/emoji
#     ├ SOUL.md        人格/语气/边界
#     ├ TOOLS.md       本地工具说明
#     └ USER.md        用户偏好
#   ~/.openclaw/agents/main/    ← 运行时家(空目录)
```

onboard 到上线 · 4 阶段真实生成 flow

极客时间

阶段 1 onboard 默认	阶段 2 向导填模型/key	阶段 3 切 workspace	阶段 4 daemon 跑+对话
~/.openclaw/ workspace/ ├ BOOTSTRAP ├ HEARTBEAT ├ IDENTITY ├ SOUL ├ TOOLS └ USER agents/main/ (空) .git/(自动 init repo)	~/.openclaw/ openclaw.json agents/main/ ├ models.json └ auth.json credentials/ telegram/ cron/	v4-prod/ openclaw/ ├ HEARTBEAT.md ├ IDENTITY.md ├ TOOLS.md ├ USER.md └ .openclaw/ state/(daemon restart 后建) SOUL/AGENTS 也会创建	v4-prod/ openclaw/state/ sessions/*.jsonl trajectory/ memory.sqlite flows/registry identity/device

└ 13-1 实操 ──▶ 13-2 实操 ─▶

极客时间黄佳老师出品

workspace 6 件套 + 全局配置 1 件

极客时间

workspace 6 件套(都在 ~/.openclaw/workspace/)

- IDENTITY.md　agent 名字 / emoji / 自我介绍
- SOUL.md　人格 / 语气 / 边界(怎么说话)
- USER.md　用户偏好(知道你是谁)
- TOOLS.md　本地工具说明(Claude/Cursor/...)
- HEARTBEAT.md　会话开始时跑的健康检查
- BOOTSTRAP.md　首次启动专用引导(可选)
-

全局配置 1 件(在 ~/.openclaw/)

- openclaw.json　是 .json 不是 .json5
-　存模型 / API Key / channels / workspace 指针
-

核心设计:身份 / 人格 / 偏好 / 工具 全部解耦

- 换 SOUL.md → 换性格;换 IDENTITY → 换名字;换 USER → 换偏好

极客时间

OpenClaw 三层隔离 · 选哪层?

OpenClaw 三层隔离(从简到强)

```
┌──────────────────────────────────────────────────────────────┐
│ ① Workspace(脚手架)        agents.defaults.workspace          │
│   IDENTITY/SOUL/USER/TOOLS  一个 bot 一个人格,本节实操目标    │
└──────────────────────────────────────────────────────────────┘
                    ↑ 升级到第二层
┌──────────────────────────────────────────────────────────────┐
│ ② Multi-Agent(同 daemon)  agents.list[] + bindings[]         │
│   多个 bot,共享模型/auth   16 节加餐,知识库 bot+闲聊 bot      │
└──────────────────────────────────────────────────────────────┘
                    ↑ 升级到第三层
┌──────────────────────────────────────────────────────────────┐
│ ③ Profile(整环境隔离)      OPENCLAW_PROFILE=work             │
│   独立数据库/config/state  工作 vs 个人,本课程不讲            │
└──────────────────────────────────────────────────────────────┘
```

口诀:一个人格用 ①,多个人格用 ②,多套环境用 ③

极客时间

极客时间

连接 Telegram

1 分钟创建 Bot · 零门槛接入

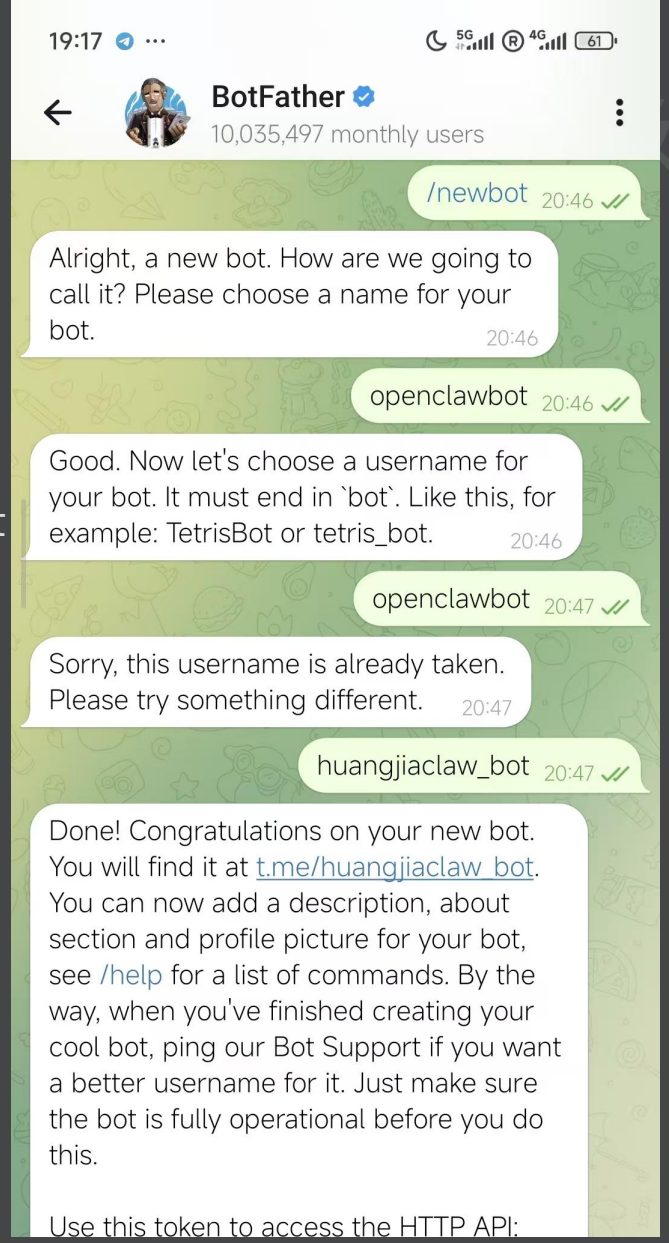
极客时间

极客时间

极客时间黄佳老师出品

创建 Telegram Bot

- **Step 1：** 在 Telegram 搜索 @BotFather
- **Step 2：** 发送 /newbot
- **Step 3：** 输入 Bot 显示名：AI 知识库助手
- **Step 4：** 输入 Bot 用户名（必须 bot 结尾）：my_kb_assistant_bot
- **Step 5：** 获得 Bot Token
-
- Bot Token 等同于密码——绝对不要提交到 Git！
-
- **安全做法：** 存入环境变量
- echo 'export TELEGRAM_BOT_TOKEN="xxx"' >> ~/.bashrc



极客时间黄佳老师出品

添加 Telegram 渠道 + DM Pairing

极客时间

terminal | bash

```bash
# 添加 Telegram 渠道
openclaw channels add --channel telegram \
  --token "$TELEGRAM_BOT_TOKEN"

# 验证渠道状态
openclaw channels list
# ┌──────────────┬──────────────────┬──────────┐
# │ Channel      │ Bot Name         │ Status   │
# ├──────────────┼──────────────────┼──────────┤
# │ telegram     │ my_kb_assistant  │ Online   │
# └──────────────┴──────────────────┴──────────┘

# 新用户首次 DM 需要配对码审批（防白嫖）
openclaw pairing list telegram          # 查看待审批
openclaw pairing approve telegram <CODE> # 审批配对

# 开发阶段可关闭（不推荐生产环境）
openclaw config set security.dmPairing false
```

极客时间

迁移知识库

V3 配置 → OpenClaw 配置（几乎零改动）

极客时间

极客时间

配置文件对比：V3 vs OpenClaw

极客时间

V3 项目结构

AGENTS.md（人格+工作指令 混合）
skills/knowledge-query.md
cost_guard.py
security.py
data/knowledge.json
.env（环境变量）

OpenClaw 项目结构

AGENTS.md（工作指令）+ SOUL.md（人格）
skills/knowledge-query.md　兼容
hooks/cost-guard.js
hooks/security-check.js
data/knowledge.json　兼容
openclaw.json5（统一配置，支持注释）

迁移映射清单

极客时间

V3 文件	OpenClaw 文件	改动量	说明
AGENTS.md（人格部分）	SOUL.md	拆分	提取性格、语气、风格
AGENTS.md（指令部分）	AGENTS.md	微调	保留职责、能力、约束
skills/*.md	skills/*.md	零改动	格式完全兼容
data/	data/	零改动	直接复制
.env	openclaw.json5	格式转换	环境变量 → JSON5（带注释）
cost_guard.py	hooks/cost-guard.js	重写	Python → JS

knowledge-query Skill 迁移

skills/knowledge-query.md | yaml

```yaml
---
name: knowledge-query
description: 当用户询问知识库中的内容时触发
allowed-tools:
  - Read
  - Grep
  - Glob
---

# 知识库查询 Skill

## 查询流程
1. 解析用户查询意图（关键词提取）
2. 在 data/knowledge.json 中搜索匹配条目
3. 按相关度排序
4. 以结构化格式返回前 5 条结果

## 输出格式
每条结果包含：标题、摘要（50字内）、来源、日期

## 注意
- description 字段是路由触发器（15 tokens）
- 完整内容只在触发后按需加载（~200 tokens）
```

本节作业

极客时间

任务 1：实操 1：安装 OpenClaw 并连接 Telegram（20 分钟）

步骤：安装 Node.js 22+ → npm install openclaw → onboard → 创建 Telegram Bot → channel add → 发送第一条消息

任务 2：实操 2：迁移知识库 Agent 配置（20 分钟）

拆分 AGENTS.md → AGENTS.md + SOUL.md → 复制 knowledge-query Skill → 测试查询

极客时间黄佳老师出品

本节小结

OpenClaw 三层架构：

Gateway（渠道接入）→ Binding（Agent 路由）→ Skills（能力执行）

2026 新特性：

Active Memory 子 Agent 自动维护跨会话记忆，无需手动管理上下文

架构哲学对比：

OpenClaw 网关优先（多渠道服务）vs Hermes 代理优先（自学习本地 Agent）

V3 → V4 迁移成本极低：Skill 文件基本零改动，只需拆分 AGENTS.md + SOUL.md

下一节：第 14 节 内容输出与分发 — 让知识库的产出到达用户手中

极客时间黄佳老师出品